

Quản lý Transaction

1 Ví dụ cơ bản: Update dữ liệu với transaction

```
// Cập nhật thông tin User
EntityManager em = emf.createEntityManager();

try {
    em.getTransaction().begin(); // bắt đầu transaction

    User user = em.find(User.class, 1); // lấy entity
    user.setName("Nguyen Van A");
    user.setEmail("a@gmail.com");

    em.getTransaction().commit(); // commit -> ghi xuống DB
} catch (Exception e) {
    if (em.getTransaction().isActive()) {
        em.getTransaction().rollback(); // rollback nếu lỗi
    }
    e.printStackTrace();
} finally {
    em.close(); // luôn đóng EntityManager
}
```

Giải thích:

- begin(): mở transaction
- commit(): xác nhận thay đổi
- rollback(): huỷ toàn bộ thay đổi nếu có lỗi

2 Ví dụ Insert (persist)

```
EntityManager em = emf.createEntityManager();

try {
    em.getTransaction().begin();

    User user = new User();
    user.setName("Tran B");
    user.setEmail("b@gmail.com");

    em.persist(user); // đưa vào persistence context

    em.getTransaction().commit();
} catch (Exception e) {
    if (em.getTransaction().isActive()) {
        em.getTransaction().rollback();
    }
} finally {
    em.close();
}
```

```
}
```

Điểm quan trọng:

- `persist()` chưa ghi DB ngay
- chỉ khi `commit()` thì mới thực sự insert

3 Ví dụ Delete

```
EntityManager em = emf.createEntityManager();

try {
    em.getTransaction().begin();

    User user = em.find(User.class, 1);
    if (user != null) {
        em.remove(user);
    }

    em.getTransaction().commit();
} catch (Exception e) {
    if (em.getTransaction().isActive()) {
        em.getTransaction().rollback();
    }
} finally {
    em.close();
}
```

4 Ví dụ thực tế trong Servlet (MVC)

```
protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {

    EntityManager em = emf.createEntityManager();

    try {
        em.getTransaction().begin();

        int id = Integer.parseInt(request.getParameter("id"));
        String name = request.getParameter("name");

        User user = em.find(User.class, id);
        user.setName(name);

        em.getTransaction().commit();

        response.sendRedirect("list"); // PRG pattern
    } catch (Exception e) {
        if (em.getTransaction().isActive()) {
            em.getTransaction().rollback();
        }
    }
}
```

```

        request.setAttribute("error", "Update failed");
        request.getRequestDispatcher("edit.jsp").forward(request, response);
    } finally {
        em.close();
    }
}

```

5 Ví dụ quan trọng: Transaction đảm bảo tính toàn vẹn

Giả sử chuyển tiền:

```

em.getTransaction().begin();

Account from = em.find(Account.class, 1);
Account to = em.find(Account.class, 2);

from.setBalance(from.getBalance() - 100);
to.setBalance(to.getBalance() + 100);

// Nếu lỗi xảy ra ở đây → rollback toàn bộ

em.getTransaction().commit();

```

Ý nghĩa:

- Nếu 1 bước lỗi → rollback → không có chuyện trừ tiền mà không cộng
- Đây là ACID (Atomicity)

6 Mẫu chuẩn khi quản lý transaction

```

EntityManager em = emf.createEntityManager();

try {
    em.getTransaction().begin();

    // business logic

    em.getTransaction().commit();
} catch (Exception e) {
    if (em.getTransaction().isActive()) {
        em.getTransaction().rollback();
    }
} finally {
    em.close();
}

```

7 Lưu ý quan trọng

- Không có transaction → JPA không ghi DB
- Mọi thao tác ghi (insert/update/delete) đều cần transaction
- find() không cần transaction (chỉ đọc)
- EntityManager không thread-safe → không dùng chung

Lý do trong JPA thì **mọi thao tác ghi (insert / update / delete) đều bắt buộc phải nằm trong transaction.**

7.1 Vì sao phải cần transaction

JPA không ghi dữ liệu xuống database ngay lập tức khi bạn gọi `persist()`, `merge()` hay `remove()`.

Các thay đổi này chỉ được đưa vào persistence context (bộ nhớ tạm).

Chỉ khi:

- transaction được `commit`
→ JPA mới thực hiện `flush` và gửi SQL thật xuống database.

Nếu không có transaction:

- sẽ xảy ra lỗi runtime (ví dụ: `TransactionRequiredException`)
- hoặc dữ liệu không được ghi xuống DB

7.2 Cụ thể từng thao tác

- insert → `persist()` → cần transaction
- update → `merge()` hoặc **dirty checking** → cần transaction
- delete → `remove()` → cần transaction

7.3 Trường hợp không cần transaction?

- Chỉ đọc dữ liệu (SELECT) thì **không bắt buộc**
- Tuy nhiên:
 - Trong thực tế (Spring, Hibernate), nhiều khi vẫn chạy trong transaction read-only để tối ưu

8 Dirty checking

Dirty checking trong JPA là cơ chế **tự động phát hiện sự thay đổi của object (entity)** và **tự sinh câu lệnh UPDATE khi transaction commit**, mà không cần lập trình viên gọi update thủ công.

8.1 Ý tưởng cốt lõi

Khi một entity được load vào `EntityManager`, JPA sẽ:

- lưu trạng thái ban đầu của entity (snapshot)
- theo dõi entity đó trong persistence context

Khi transaction kết thúc (commit):

- JPA so sánh trạng thái hiện tại với snapshot
- nếu có thay đổi → sinh SQL UPDATE

8.2 Ví dụ cụ thể

```
EntityManager em = emf.createEntityManager();
EntityTransaction tx = em.getTransaction();

tx.begin();

// Lấy entity từ DB → entity này đang được manage
Student s = em.find(Student.class, 1);

// Chỉ cần thay đổi giá trị
s.setName("Bình");

// KHÔNG cần gọi em.merge() hay update gì cả

tx.commit(); // → JPA tự phát hiện thay đổi và UPDATE DB
```

8.3 Điều kiện để dirty checking hoạt động

- Entity phải ở trạng thái managed (đang được EntityManager quản lý)
- Phải nằm trong transaction
- Thay đổi xảy ra trước khi commit

Nếu entity:

- detached → JPA không theo dõi → không dirty checking
→ phải dùng `merge()`

8.4 Khi nào JPA thực hiện dirty checking

- Trước khi `commit`
- Hoặc khi `flush()` được gọi

8.5 Lợi ích

- Giảm code (không cần viết update)
- Tránh sai sót khi quên gọi update
- Code gần với OOP hơn (chỉ cần sửa object)

8.6 Lưu ý dễ nhầm

- `merge()` không phải là update trực tiếp, mà là:
 - copy dữ liệu từ object detached vào managed entity
- Dirty checking mới là thứ thực sự tạo câu lệnh UPDATE